# FireSense

## Manual de Configuración

*Guía completa paso a paso para instalar y conectar tu nodo IoT RAK 4631!*

| | | | |
|---|---|---|---|
| **Versión** | **1.0** | **Plataforma** | **FireSense IoT** |
| **Idioma** | **Español** | **Hardware** | **RAK4631 + RAK7289V2** |
| **Fecha** | **2026** | **Protocolo** | **LoRaWAN EU868 OTAA** |

# ÍNDICE


1. Requisitos de hardware y software
2. Instalar y configurar ChirpStack (servidor propio) — Opción A
3. Configurar el entorno Arduino IDE
4. Obtener claves OTAA y programar el firmware RAK4631
5. Configurar el gateway RAK7289V2
6. Registrar el nodo en ChirpStack
7. ChirpStack — Opción B: Servidor FireSense Cloud
8. Configurar Node-RED (importar flow automáticamente)
9. Integración InfluxDB directa a ChirpStack
10. Configurar Grafana — Dashboard visual
11. Vincular el nodo a la plataforma FireSense
12. Verificación y resolución de problemas
13. JSON Node-RED para importar


✓ ORDEN DE CONFIGURACIÓN RECOMENDADO: Primero instala ChirpStack (sección 2), después programa el firmware (sección 4) con las claves que obtuviste de ChirpStack, después configura el gateway (sección 5) con la IP de tu servidor ya conocida.

# 1 Requisitos de hardware y software

### Hardware necesario:

| Componente | Modelo | Función |
|---|---|---|
| Microcontrolador | RAK4631 (nRF52840 + SX1262) | Nodo sensor LoRaWAN |
| Placa base | RAK19007 WisBlock Base | Conecta todos los módulos sin soldadura |
| Sensor temp/humedad aire | RAK1901 (SHTC3) | Temperatura y humedad del aire (I2C) |
| Sensor humedad suelo | RAK12035 | Humedad del suelo 0-100% (I2C) |
| Gateway LoRaWAN | RAK7289V2 | Puente radio LoRa — internet |
| Batería | LiPo 3.7V (2x18650 recomendado) | Alimentación autónoma meses |

### Software necesario:

| Software | Versión | Descarga |
|---|---|---|
| Arduino IDE | 2.x o superior | arduino.cc/en/software |
| BSP RAKwireless nRF | v1.3.3+ | Via Arduino IDE Board Manager |
| Driver USB CH341 | Último | wch-ic.com/downloads/CH341SER_EXE.html |

# 2 Instalar ChirpStack con Docker — Opción A: Servidor propio

✓ Instalamos ChirpStack PRIMERO porque necesitamos su IP y las claves OTAA antes de programar el firmware y configurar el gateway. Si usas el servicio Cloud de FireSense, salta a la sección 7.

## 2.1 Requisitos del servidor

- Ubuntu 22.04 LTS o Debian 12 (recomendado)
- Mínimo 2GB RAM, 20GB de espacio en disco
- Puertos accesibles desde el gateway: 8080 (UI ChirpStack), 1883 (MQTT)
- IP fija o dominio conocido (necesitarás esta IP en los pasos siguientes)

## 2.2 Instalar Docker

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker

# Verificar instalación:
docker --version
docker compose version
```

## 2.3 Crear estructura de directorios

```
mkdir -p ~/chirpstack/{chirpstack,mosquitto,postgres,redis}
mkdir -p ~/chirpstack/chirpstack/regions
cd ~/chirpstack
```

## 2.4 Crear docker-compose.yml

```
cat > docker-compose.yml << 'EOF'
services:
  chirpstack:
    image: chirpstack/chirpstack:4
    container_name: chirpstack
    restart: unless-stopped
    command: -c /etc/chirpstack
    volumes:
      - ./chirpstack:/etc/chirpstack
    depends_on: [postgres, mosquitto, redis]
    ports:
      - "8080:8080"
    networks: [chirpstack-net]

  postgres:
    image: postgres:14-alpine
    container_name: chirpstack-postgres
    restart: unless-stopped
    environment:
      - POSTGRES_PASSWORD=chirpstack
      - POSTGRES_USER=chirpstack
      - POSTGRES_DB=chirpstack
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks: [chirpstack-net]

  redis:
    image: redis:7-alpine
    container_name: chirpstack-redis
    restart: unless-stopped
    volumes: [redis-data:/data]
    networks: [chirpstack-net]

  mosquitto:
    image: eclipse-mosquitto:2
    container_name: chirpstack-mqtt
    restart: unless-stopped
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto:/mosquitto/config
    networks: [chirpstack-net]

networks:
  chirpstack-net:
    driver: bridge

volumes:
  postgres-data:
  redis-data:
EOF
```

## 2.5 Crear chirpstack.toml

```
cat > chirpstack/chirpstack.toml << 'EOF'
[logging]
  level = "info"

[postgresql]
  dsn = "postgres://chirpstack:chirpstack@postgres/chirpstack?sslmode=disable"

[redis]
  servers = ["redis://redis/"]

[network]
  net_id = "000000"
  enabled_regions = ["eu868"]

[api]
  bind = "0.0.0.0:8080"
```

```
  secret = "CAMBIA_POR_UNA_CLAVE_LARGA_Y_SEGURA"

[integration]
  enabled = ["mqtt"]
  [integration.mqtt]
    server = "tcp://mosquitto:1883"
    json = true
EOF
```

## 2.6 Crear eu868.toml — CRÍTICO para ver el gateway

⚠ **Atención:** Sin la sección [regions.gateway] ChirpStack NO se suscribirá a los topics MQTT del gateway y nunca lo verá Online. Este fichero es imprescindible.

```
cat > chirpstack/regions/eu868.toml << 'EOF'
[[regions]]
  id = "eu868"
  description = "EU868"

  [regions.gateway]
    force_gws_private = false
    [regions.gateway.backend]
      enabled = "mqtt"
      [regions.gateway.backend.mqtt]
        event_topic   = "eu868/gateway/+/event/+"
        command_topic = "eu868/gateway/{{ gateway_id }}/command/{{ command }}"
        server        = "tcp://mosquitto:1883"
        clean_session = false
        qos           = 0

  [regions.network]
    installation_margin = 10
    rx_window = 0
    rx1_delay = 1
    rx2_dr = 0
    rx2_frequency = 869525000
    min_dr = 0
    max_dr = 5
    uplink_max_eirp = 16

  [[regions.network.extra_channels]]
    frequency = 867100000
    min_dr = 0
    max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867300000
    min_dr = 0
    max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867500000
    min_dr = 0
    max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867700000
    min_dr = 0
    max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867900000
    min_dr = 0
    max_dr = 5
EOF
```

## 2.7 Crear configuración de Mosquitto

```
cat > mosquitto/mosquitto.conf << 'EOF'
listener 1883
allow_anonymous true
persistence true
```

```
persistence_location /mosquitto/data/
log_dest stdout
EOF
```

## 2.8 Iniciar ChirpStack

```
cd ~/chirpstack
docker compose up -d

# Ver logs (espera ver "starting api server"):
docker compose logs -f chirpstack

# Verificar que todos están healthy:
docker compose ps
```

ℹ ChirpStack disponible en http://TU_IP:8080 — Usuario: admin — Contraseña: admin. Cámbiala inmediatamente.

### ¿Cómo saber cuál es TU_IP?

Ejecuta este comando en tu servidor para ver la IP:

```
# En Linux/Mac:
ip addr show | grep "inet " | grep -v 127.0.0.1
# Ejemplo de resultado: inet 192.168.1.50/24 — tu IP es 192.168.1.50

# En Windows (PowerShell):
ipconfig | findstr IPv4
```

Anota esta IP. La usarás en el gateway (paso 5.3) y en Node-RED (paso 8.4).

## 2.9 Crear Device Profile

ChirpStack → Tenants → Default → Device Profiles → Add device profile:

| Parámetro | Valor |
|---|---|
| Name | firesense-node-otaa |
| Region | EU868 |
| MAC version | LoRaWAN 1.0.3 |
| Regional parameters revision | A |
| ADR algorithm | Default ADR |
| Expected uplink interval | 3600 segundos |
| Device supports OTAA | Activado ✓ |

## 2.10 Añadir codec JavaScript (7 bytes)

Device Profile → pestaña Codec → JavaScript functions → pega este código:

```
function decodeUplink(input) {
  var b = input.bytes;
  if (b.length < 7) return { data: {} };
  var t = (b[0] << 8) | b[1];
  if (t > 32767) t -= 65536;  // temperatura negativa
  var h = (b[2] << 8) | b[3];
  var bat = (b[5] << 8) | b[6];
```

```
  return { data: {
    temperatura:   parseFloat((t / 100).toFixed(2)),
    humedad_aire:  parseFloat((h / 100).toFixed(2)),
    humedad_suelo: b[4],
    bateria_mv:    bat
  }};
}
```

⚠ **Atención:** El payload es de 7 bytes. Usa siempre b.length < 7. Si pones == 5 el codec no decodificará nada.

### 2.11 Crear Application

ChirpStack → Applications → Add application → Name: firesense-nodes → Submit.

✓ Anota la IP de tu servidor ChirpStack (la misma donde accediste al panel). La necesitarás en los pasos 3, 5 y 8.

## 3 Configurar el entorno Arduino IDE

### 3.1 Instalar Arduino IDE

Descarga Arduino IDE 2.x desde arduino.cc/en/software e instálalo en tu sistema operativo.

### 3.2 Añadir URL de RAKwireless

Arduino IDE → File → Preferences → campo "Additional boards manager URLs" → añade:

```
https://raw.githubusercontent.com/RAKWireless/RAKwireless-Arduino-BSP-Index
/main/package_rakwireless_index.json
```

### 3.3 Instalar BSP RAKwireless

Tools → Board → Boards Manager → busca "RAK" → instala RAKwireless nRF Boards (v1.3.3+). Tarda varios minutos.

### 3.4 Instalar librerías

Tools → Manage Libraries → instala estas 4:

- LoRaWan-RAK4630 — gestión LoRaWAN OTAA
- Adafruit SHTC3 Library (v1.0.2+) — sensor temperatura/humedad aire
- RAK12035_SoilMoisture — sensor humedad suelo
- Adafruit Unified Sensor — dependencia de SHTC3 (requerida)

### 3.5 Seleccionar placa y puerto

Tools → Board → RAKwireless nRF Boards → WisBlock RAK4631. Conecta el RAK4631 por USB → Tools → Port → selecciona el puerto (COMx en Windows, /dev/ttyUSB0 en Linux).

# 4 Obtener claves OTAA y programar el firmware RAK4631

✓ Ahora que ChirpStack está funcionando, obtenemos las claves OTAA directamente desde su interfaz y las ponemos en el firmware.

## 4.1 Obtener el Device EUI del nodo

El Device EUI está impreso en la etiqueta adhesiva del módulo RAK4631. Formato en la etiqueta: AC:1F:09:FF:FE:24:54:24

⚠ **Atención:** El firmware necesita el DevEUI en formato hexadecimal C++, NO con dos puntos. Convierte así: cada par separado por coma con prefijo 0x.

| Formato en etiqueta | Formato en firmware |
|---|---|
| AC:1F:09:FF:FE:24:54:24 | {0xAC, 0x1F, 0x09, 0xFF, 0xFE, 0x24, 0x54, 0x24} |

## 4.2 Obtener la Application Key desde ChirpStack

La Application Key (AppKey) es una clave secreta de 16 bytes. Hay dos opciones:

OPCIÓN A — Generar nueva AppKey en ChirpStack (recomendado):

- ChirpStack → Applications → firesense-nodes → Add device
- Rellena: Name (ej: rak-nodo-01), Device EUI (sin dos puntos)
- Device Profile: firesense-node-otaa → Submit
- Ve a la pestaña OTAA Keys → haz clic en el icono de dados para GENERAR una AppKey aleatoria
- Copia la AppKey generada → la usarás en el firmware

OPCIÓN B — Usar AppKey ya grabada en el firmware:

- Si el nodo ya tiene una AppKey en el firmware, anótala
- En ChirpStack → OTAA Keys → introduce esa misma AppKey manualmente

✓ La AppKey del firmware y la de ChirpStack DEBEN coincidir exactamente. Si no coinciden, el nodo nunca podrá hacer el Join OTAA.

## 4.3 Código completo del firmware

Crea un nuevo sketch en Arduino IDE. Sustituye las claves por las tuyas:

```
#include <Arduino.h>
#include <LoRaWan-RAK4630.h>
#include <SPI.h>
#include <Wire.h>
#include "Adafruit_SHTC3.h"
#include <RAK12035_SoilMoisture.h>

# SUSTITUIR: DevEUI de la etiqueta (formato 0xXX)
uint8_t nodeDevEui[] = {0xAC, 0x1F, 0x09, 0xFF, 0xFE, 0x24, 0x54, 0x24};
uint8_t nodeAppEui[] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
```

```
# SUSTITUIR: AppKey obtenida de ChirpStack (paso 4.2)
uint8_t nodeAppKey[] = {0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX,
                        0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX};

Adafruit_SHTC3 shtc3;
RAK12035 soil_sensor;
bool joined = false;
#define vbat_pin        A0
#define VBAT_MV_PER_LSB 1.279f

void lorawan_rx_handler(lmh_app_data_t *d) {}
void lorawan_has_joined_handler(void) { joined = true; }
void lorawan_join_failed_handler(void) {}
void lorawan_confirm_class_handler(DeviceClass_t C) {}
static lmh_callback_t lora_callbacks = {
  BoardGetBatteryLevel, BoardGetUniqueId, BoardGetRandomSeed,
  lorawan_rx_handler, lorawan_has_joined_handler,
  lorawan_confirm_class_handler, lorawan_join_failed_handler };
static lmh_param_t lora_param_init = {
  LORAWAN_ADR_ON, DR_3, LORAWAN_PUBLIC_NETWORK, 8,
  LORAWAN_DEFAULT_DATARATE, CLASS_A };

float readVBAT() { return analogRead(vbat_pin) * VBAT_MV_PER_LSB; }
uint8_t mvToPercent(float mv) {
  if (mv>=4200) return 100; if (mv>=4000) return 85;
  if (mv>=3800) return 60;  if (mv>=3600) return 30;
  if (mv>=3400) return 10;  return 0; }

void setup() {
  pinMode(WB_IO2, OUTPUT);
  digitalWrite(WB_IO2, HIGH); # CRITICO: enciende alimentacion sensores
  delay(500);
  analogReference(AR_INTERNAL_3_0);
  analogReadResolution(12);
  delay(5); readVBAT(); readVBAT();
  Wire.begin(); shtc3.begin(); soil_sensor.begin();
  lora_rak4630_init();
  lmh_setDevEui(nodeDevEui);
  lmh_setAppEui(nodeAppEui);
  lmh_setAppKey(nodeAppKey);
  lmh_init(&lora_callbacks, lora_param_init, true, CLASS_A, LORAMAC_REGION_EU868);
  lmh_join();
}

void loop() {
  if (!joined) { delay(1000); return; }
  sensors_event_t hum, temp;
  shtc3.getEvent(&hum, &temp);
  uint8_t moisture = 0;
  soil_sensor.get_sensor_moisture(&moisture);
  float bat = (readVBAT()+readVBAT()+readVBAT())/3.0f;
  uint16_t batInt = (uint16_t)bat;
  int16_t  t = (int16_t)(temp.temperature * 100);
  uint16_t h = (uint16_t)(hum.relative_humidity * 100);
  # Payload 7 bytes: temp(2)+hum_aire(2)+hum_suelo(1)+bat_mv(2)
  uint8_t payload[7];
  payload[0]=(uint8_t)(t>>8);      payload[1]=(uint8_t)t;
  payload[2]=(uint8_t)(h>>8);      payload[3]=(uint8_t)h;
  payload[4]=moisture;
  payload[5]=(uint8_t)(batInt>>8); payload[6]=(uint8_t)batInt;
  lmh_app_data_t d; d.port=1; d.buffsize=7; d.buffer=payload;
  lmh_send(&d, LMH_UNCONFIRMED_MSG);
  delay(3600000); # Dormir 1 hora
}
```

## 4.4 Subir el firmware

Haz clic en Upload (→). Tardará 1-2 minutos. Abre Monitor Serie a 115200 baudios. Verás el Join Request y Join Accept si ChirpStack está corriendo y el gateway está Online.

⚠ **Atención:** Si WB_IO2 da error, el BSP RAKwireless no está instalado. Repite el paso 3.3.

✓ Si ves JoinRequest en el Monitor Serie pero NO ves JoinAccept, es completamente normal en este punto. El gateway todavía no está configurado. Continúa con la sección 5 y el JoinAccept llegará cuando el gateway esté Online.

## 5 Configurar el gateway RAK7289V2

✓ Ahora ya tienes la IP de tu servidor ChirpStack (del paso 2). La usarás aquí como MQTT Broker Address.

### 5.1 Conectar el gateway a internet

Conecta por Ethernet a tu router o activa la 4G integrada. Espera 2 minutos (LED verde fijo).

### 5.2 Acceder a la interfaz web

```
URL:        http://192.168.1.1
Usuario:    root
Contraseña: root
```

ℹ Si no accedes, conecta Ethernet directo entre PC y gateway. Tu PC obtendrá IP 192.168.1.x automáticamente.

### 5.3 Configurar MQTT Bridge

LoRa → LoRa Network → Network Settings:

| Parámetro | Valor |
|---|---|
| Protocol | LoRa Gateway MQTT Bridge |
| MQTT Protocol Version | MQTT for ChirpStack 4.x (Protobuf) |
| MQTT Broker Address | IP DE TU SERVIDOR CHIRPSTACK (del paso 2) |
| MQTT Broker Port | 1883 |
| SSL/TLS | None (desactivado) |
| User Authentication | Desactivado |

### 5.4 WireGuard VPN — Solo si red privada

⚠ **Atención:** Solo si el gateway está en red privada sin acceso directo al servidor. Si tienes internet directo, salta al paso 5.5.

**Extensions → RAK WireGuard → Install. Configura con los datos de tu administrador:**

- Address: IP del gateway en el túnel (ej: 10.0.0.28/24)
- Endpoint: IP del servidor VPN:puerto (ej: servidor.empresa.com:443)
- AllowedIPs: Red del servidor (ej: 10.2.0.0/16)
- PersistentKeepalive: 25000 ms

### 5.5 Verificar gateway Online

**ChirpStack → Gateways:** punto verde y "Last seen" hace pocos segundos.

Si aparece Offline: revisa la IP del MQTT Broker, el puerto y la conectividad de red.

## 6 Registrar el nodo en ChirpStack

✓ Verifica en ChirpStack → Applications → firesense-nodes → tu device → LoRaWAN Frames que ves JoinRequest + JoinAccept. Si ya los ves, el nodo está registrado y funcionando correctamente. Si el device no existe todavía, sigue los pasos de abajo.

### 6.1 Añadir el device en ChirpStack

ChirpStack → Applications → firesense-nodes → Add device:

- Name: rak-nodo-01 (o el nombre que prefieras)
- Device EUI: el de la etiqueta del RAK4631, SIN dos puntos (ej: ac1f09fffe245424)
- Device Profile: firesense-node-otaa
- → Submit

### 6.2 Configurar OTAA Keys

Pestaña OTAA Keys → introduce la Application Key. Debe ser EXACTAMENTE la misma que pusiste en el firmware (paso 4.2). → Submit.

### 6.3 Verificar Join OTAA

Enciende el nodo RAK4631. En ChirpStack → Device → LoRaWAN Frames, deberías ver:

```
JoinRequest  ←  el nodo pide unirse
JoinAccept   →  ChirpStack acepta la conexión
             (aparece en pocos segundos si el gateway está Online)
```

Si ves JoinRequest pero no JoinAccept: la AppKey no coincide entre firmware y ChirpStack.

## 7 ChirpStack — Opción B: Servidor FireSense Cloud

Si contratas el servicio Cloud de FireSense, el servidor ya está configurado. Recibirás credenciales por email.

### 7.1 Acceder al panel

```
URL:        https://chirpstack.firesense.io
Usuario:    tu_email@empresa.com
Contraseña: (la recibida por email)
```

### 7.2 Registrar nodo y gateway

El Device Profile ya está creado. Ve a Applications → firesense-nodes → Add device. Sigue los pasos 6.1 a 6.3.

Para el gateway, usa como MQTT Broker Address: mqtt.firesense.io, puerto 1883.

## 8 Configurar Node-RED

Node-RED recibe los datos de ChirpStack y los guarda en InfluxDB. Necesitas tener la IP de tu servidor InfluxDB y su token de acceso.

### 8.1 Obtener el token de InfluxDB

Accede a InfluxDB en http://TU_IP:8086 → Data → API Tokens → Generate API Token → All Access Token. Copia el token generado.

### 8.2 Acceder a Node-RED

```
http://TU_IP:1880/nodered/
```

### 8.3 Importar el flow

Menú ≡ → Import → pega el JSON de la sección 13 → Import → Deploy (botón rojo).

### 8.4 Configurar credenciales

Doble clic en nodo "InfluxDB":

- URL: http://influxdb:8086 (o http://TU_IP_INFLUXDB:8086)
- Token: el token copiado en el paso 8.1
- Organization: tu organización
- Bucket: sensor_data

Doble clic en nodo "MQTT ChirpStack" → Edit Broker:

⚠ **Atención:** Cambia 'localhost' por la IP real de tu servidor ChirpStack. Si Node-RED y ChirpStack están en el mismo servidor, puedes dejarlo como 'localhost' o 'mosquitto'.

→ Deploy para guardar.

## 9 Integración InfluxDB directa a ChirpStack

Alternativa a Node-RED: ChirpStack puede enviar datos directamente a InfluxDB sin intermediarios.

ℹ Puedes usar Node-RED (sección 8) O esta integración directa. No es necesario usar las dos. La integración directa es más simple; Node-RED permite más procesamiento avanzado.

### 9.1 Activar integración

ChirpStack → Applications → firesense-nodes → pestaña ... → InfluxDB → botón +:

| Parámetro | Valor |
|---|---|
| Endpoint | http://TU_IP:8086 |
| Version | InfluxDB v2 |
| Token | Token de administrador de InfluxDB |
| Organization | Tu organización |
| Bucket | sensor_data |

→ Submit. ChirpStack escribirá automáticamente todos los campos del codec en InfluxDB.

## 10 Configurar Grafana — Dashboard visual

### 10.1 Acceder a Grafana

```
URL:        http://TU_IP:3000
Usuario:    admin
Contraseña: admin  (cambiar en primer acceso)
```

### 10.2 Añadir InfluxDB como Data Source

Configuration → Data Sources → Add data source → InfluxDB:

| Parámetro | Valor |
|---|---|
| Name | InfluxDB FireSense |
| Query Language | Flux |
| URL | http://influxdb:8086 (o http://TU_IP:8086) |
| Organization | Tu organización |
| Token | Token de administrador InfluxDB |
| Default Bucket | sensor_data |

→ Save & Test → debe aparecer "datasource is working".

### 10.3 Activar embedding para la web FireSense

Para mostrar paneles Grafana dentro del mapa FireSense, añade en el docker-compose de Grafana:

```
environment:
  - GF_AUTH_ANONYMOUS_ENABLED=true
  - GF_AUTH_ANONYMOUS_ORG_ROLE=Viewer
  - GF_SECURITY_ALLOW_EMBEDDING=true
```

**Reinicia:** docker compose restart grafana

### 10.4 Importar dashboard FireSense

Grafana → Dashboards → Import → pega este JSON del dashboard:

```
{"title":"FireSense — Nodes
IoT","uid":"firesense-main","tags":["firesense","iot"],"timezone":"Europe/Madrid",
"panels":[
  {"id":1,"type":"timeseries","title":"Temperatura
(°C)","gridPos":{"x":0,"y":0,"w":12,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -24h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"temperatura\")"}],
   "fieldConfig":{"defaults":{"unit":"celsius","color":{"mode":"palette-classic"}}}},
  {"id":2,"type":"timeseries","title":"Humedad Aire
(%)","gridPos":{"x":12,"y":0,"w":12,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -24h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"humedad_aire\")"}],
   "fieldConfig":{"defaults":{"unit":"percent","min":0,"max":100}}},
  {"id":3,"type":"timeseries","title":"Humedad Suelo
(%)","gridPos":{"x":0,"y":8,"w":8,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -24h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"humedad_suelo\")"}],
   "fieldConfig":{"defaults":{"unit":"percent","min":0,"max":100}}},
  {"id":4,"type":"gauge","title":"Batería (%)","gridPos":{"x":8,"y":8,"w":8,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -1h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"bateria_mv\")\n  |> last()"}],

"fieldConfig":{"defaults":{"unit":"mV","thresholds":{"steps":[{"color":"red","value":0}
,{"color":"yellow","value":3600},{"color":"green","value":3800}]}}}},
  {"id":5,"type":"stat","title":"RSSI","gridPos":{"x":16,"y":8,"w":8,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -1h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"rssi\")\n  |> last()"}],

"fieldConfig":{"defaults":{"unit":"dBm","thresholds":{"steps":[{"color":"red","value":-
120},{"color":"yellow","value":-100},{"color":"green","value":-80}]}}}}
],"time":{"from":"now-24h","to":"now"},"refresh":"1m","schemaVersion":38}
```

Después de importar → en cada panel → Edit → Query → Data source → selecciona "InfluxDB FireSense" → Save dashboard.

## 11 Vincular el nodo a la plataforma FireSense

✓ Este es el último paso. El nodo ya envía datos a InfluxDB. Ahora lo vinculas a tu cuenta FireSense para que aparezca en el mapa.

### 11.1 Iniciar sesión en FireSense

```
URL: https://TU_DOMINIO/FireSense/login.html
```

### 11.2 Abrir el asistente y vincular el Device EUI

En el dashboard → barra "Telemetria en Viu" → botón + AFEGIR NODE (arriba a la derecha).

El asistente tiene 4 pasos. En el paso 4, introduce el Device EUI en formato de 16 caracteres hexadecimales SIN dos puntos:

| Formato correcto | Formato INCORRECTO |
|---|---|
| ac1f09fffe245424 | AC:1F:09:FF:FE:24:54:24 |

→ Vincular node. El nodo aparecerá en el mapa automáticamente.

ℹ Puedes añadir tantos nodos como quieras repitiendo este proceso. Cada nodo aparece en el mapa con ID automático (RAK-01, RAK-02...) con sus datos de temperatura, humedad y batería.

## 12 Verificación y resolución de problemas

**Lista de verificación:**

| OK | Verificación |
|---|---|
| [ ] | ChirpStack accesible en http://TU_IP:8080 |
| [ ] | Gateway aparece Online en ChirpStack (punto verde en Gateways) |
| [ ] | Nodo hace Join OTAA (JoinRequest + JoinAccept en Device → LoRaWAN Frames) |
| [ ] | Datos decodificados en ChirpStack → Device → Events |
| [ ] | Node-RED recibe datos (nodo debug muestra objeto con campos) |
| [ ] | InfluxDB contiene datos en bucket sensor_data |
| [ ] | Grafana muestra gráficas (Data Source funciona) |
| [ ] | Nodo vinculado y visible en el mapa FireSense |

**Problemas frecuentes:**

| Problema | Solución |
|---|---|
| Gateway Offline en ChirpStack | Verificar IP del MQTT Broker (debe ser la IP del servidor ChirpStack). Verificar puerto 1883. |
| Nodo no hace JoinAccept | AppKey en firmware y ChirpStack no coinciden. Verificar byte a byte. |
| JoinRequest pero no JoinAccept | El gateway no está Online o la AppKey no coincide. |
| Sensores devuelven 0 o 255 | Falta digitalWrite(WB_IO2, HIGH) en setup(). Es la línea más importante del firmware. |
| Codec no decodifica datos | Verificar que usa b.length < 7 (payload de 7 bytes). No usar == 5. |
| No hay datos en InfluxDB | Verificar token InfluxDB en Node-RED. El tag en el JSON debe ser dev_eui (no node_id). |
| Nodo no aparece en el mapa | Verificar que el DevEUI registrado en FireSense coincide con el que usa el nodo (sin mayúsculas ni dos puntos). |

## 13 JSON Node-RED — Flow completo para importar

✓ IMPORTANTE: Después de importar el JSON, hay DOS configuraciones que debes cambiar: (1) En el nodo 'MQTT ChirpStack' — doble clic — Edit Broker — Server: cambia 'localhost' por la IP de tu servidor ChirpStack. (2) En el nodo 'InfluxDB' — doble clic — cambia la URL, token y organización. El resto del JSON no necesita modificaciones.

```
[
  {
    "id": "firesense-main-tab",
    "type": "tab",
    "label": "FireSense LoRaWAN",
    "disabled": false
  },
  {
    "id": "chirpstack-mqtt-in",
    "type": "mqtt in",
    "z": "firesense-main-tab",
    "name": "ChirpStack MQTT",
    "topic": "application/+/device/+/event/up",
    "qos": "0",
    "datatype": "json",
    "broker": "chirpstack-broker-cfg",
    "x": 130, "y": 100,
    "wires": [["fn-decode-payload"]]
  },
  {
    "id": "fn-decode-payload",
    "type": "function",
    "z": "firesense-main-tab",
    "name": "Decodificar 7 bytes",
    "func": "var b=msg.payload.data;if(!b)return null;var
bytes=Buffer.from(b,'base64');if(bytes.length<7)return null;var
t=(bytes[0]<<8)|bytes[1];if(t>32767)t-=65536;var h=(bytes[2]<<8)|bytes[3];var
bat=(bytes[5]<<8)|bytes[6];msg.payload={measurement:'espurna_sensors',tags:{dev_eui:msg.payload.deviceInfo.
devEui,device_name:msg.payload.deviceInfo.deviceName},fields:{temperatura:parseFloat((t/100).toFixed(2)),hu
medad_aire:parseFloat((h/100).toFixed(2)),humedad_suelo:bytes[4],bateria_mv:bat,rssi:(msg.payload.rxInfo&&m
sg.payload.rxInfo[0])?msg.payload.rxInfo[0].rssi:0,snr:(msg.payload.rxInfo&&msg.payload.rxInfo[0])?msg.payl
oad.rxInfo[0].snr:0,fcnt:msg.payload.fCnt||0},timestamp:new Date().getTime()*1000000};return msg;",
    "x": 330, "y": 100,
    "wires": [["influxdb-write-out","debug-datos"]]
  },
  {
    "id": "influxdb-write-out",
    "type": "influxdb out",
    "z": "firesense-main-tab",
    "influxdb": "influxdb-v2-cfg",
    "name": "Guardar en InfluxDB",
    "measurement": "espurna_sensors",
    "precision": "ns",
    "x": 560, "y": 80,
    "wires": []
  },
  {
    "id": "debug-datos",
    "type": "debug",
    "z": "firesense-main-tab",
    "name": "Debug",
    "active": true,
    "complete": "payload",
    "x": 560, "y": 120,
    "wires": []
  },
  {
    "id": "chirpstack-broker-cfg",
    "type": "mqtt-broker",
    "name": "ChirpStack MQTT",
    "broker": "localhost",
    "port": "1883",
    "clientid": "nodered-firesense",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": "4",
    "keepalive": "60",
    "cleansession": true
  },
  {
    "id": "influxdb-v2-cfg",
    "type": "influxdb",
    "hostname": "localhost",
    "port": "8086",
    "protocol": "http",
    "name": "InfluxDB v2 FireSense",
    "influxdbVersion": "2.0",
    "url": "http://localhost:8086",
    "rejectUnauthorized": false
  }
]
```


*FireSense IoT Platform — Institut Tecnologic de Barcelona — ASIX2c — 2025/2026*
*Soporte: soporte@firesense.io | Documentación: docs.firesense.io*